A Shot-Classification Agent for CricShot10k

How it is built, explained simply

CricShot project

In one paragraph

The agent watches one broadcast clip of a batter and says which of the 15 cricket shots it is. It first finds the batter and cuts out the frames that matter. If it cannot see the batter well enough, it says so instead of guessing. Otherwise one small, quick model gives an answer. If that model is sure, the agent answers straight away. If it is not sure, several bigger models that look at the clip in different ways are asked, and their answers are combined by a weighted vote. The agent always explains its answer: the top three shots, which models agreed, and the frames that mattered.

Every step is an ordinary function around a trained model. There is no large language model inside: the agent is a fixed flowchart, built with LangGraph so it can plug into CricketMind as one node. Figure 1 shows the whole agent.

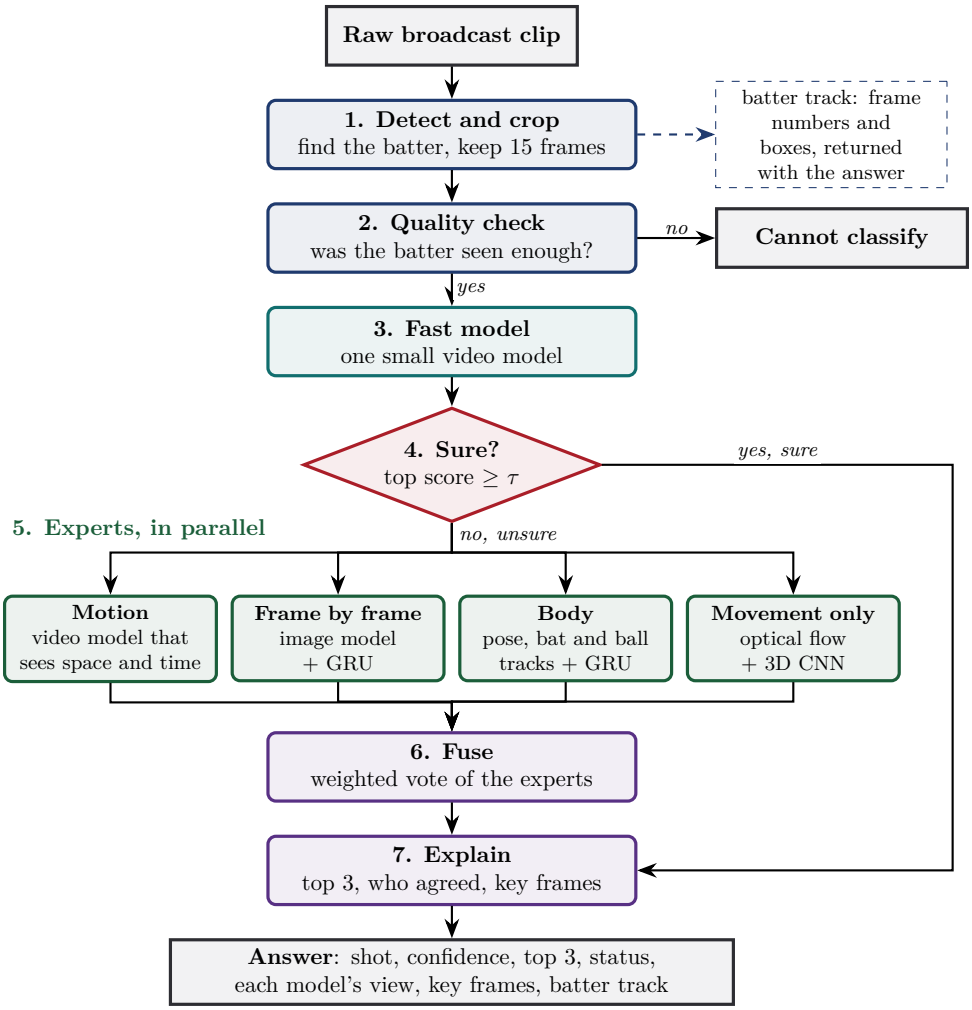


Figure 1: The agent. Blue prepares the clip, red decides, teal is the one quick model, green are the experts that are only asked when the quick model is unsure, and violet combines and explains. A clip takes the short path (1, 2, 3, 4, 7) whenever the quick model is sure.

1 Each step, simply

1. Detect and crop

A detector (YOLO, trained by the CricShot10k authors) finds the batter in every frame. Around the batter the agent cuts a square, so the batter fills the picture and the crowd mostly does not. It keeps the main camera shot, drops repeated frames, and picks 15 frames spread evenly over the stroke.
In: a raw video clip **Out:** 15 cropped frames of 224×224 pixels, and the *batter track*: for every kept frame, its number in the raw clip and the box that was cut out
Why it is there: every model in CricShot was trained on frames made exactly this way. Frames made any other way look different to the models, and they answer worse without any error message. The batter track lets CricketMind run its own pose measurements on the very same frames.

2. Quality check

If the batter was found in too few frames, the agent stops and returns *cannot classify*. It uses the same rule CricShot's preprocessing uses to drop a clip.
In: the 15 frames and how often the batter was found **Out:** go on, or stop
Why it is there: a confident guess about a clip with no batter in it is worse than an honest "I can't tell".

3. The fast model

One small video model looks at the 15 frames and gives a score to each of the 15 shots. The scores are turned into probabilities that add up to 1 (see Section 2).
In: 15 frames **Out:** 15 probabilities, one per shot
Why it is there: most clips are easy. A small model answers them quickly, so the big models only run when they are needed.
Candidates from Phase 3: X3D-M or R(2+1)D-18, both small and quick. The final choice is made on validation data in Phase 4.

4. The confidence gate

The agent looks at the fast model's highest probability. If it is at least a threshold τ (for example 0.9), the agent is sure and goes straight to step 7. If not, it asks the experts.
In: 15 probabilities **Out:** "sure" or "unsure"
Why it is there: this is the agent's main decision: answer now, or spend more effort. A higher τ means more clips go to the experts: slower, but more careful.

5. The experts

Several stronger models each give their own 15 probabilities, all at the same time. They are chosen because they look at the clip in different ways:

- **Motion:** a video model that sees space and time together (candidates: MViT-v2-S, SlowFast).
- **Frame by frame:** an image model describes each frame, and a GRU reads the 15 descriptions in order (ConvNeXt-T + GRU).
- **Body:** the batter's 33 body points, plus the bat's and the ball's paths, read by a GRU (pose GRU, or bat + ball + pose).
- **Movement only:** optical flow, how every pixel moved between frames, read by a 3D CNN (flow r3d_18).

In: the 15 frames (and the pose and flow made from them) **Out:** one set of 15 probabilities per expert
Why it is there: models that see differently make *different* mistakes, so when they vote, one model's mistake can be outvoted by the others. Phase 3 already found that averaging a body-pose model with a picture model beats either one alone.

6. Fuse

The experts' probabilities are averaged, with a weight for each expert (Section 2). Better experts get bigger weights. The shot with the highest combined probability is the answer.
In: one set of probabilities per expert **Out:** one combined set
Why it is there: a weighted vote is simple, has very few numbers to learn, and so cannot memorise the training clips. In Phase 3, a plain average beat a fusion layer that had to be trained.

7. Explain

The agent writes down the top three shots with their probabilities, which models agreed with the answer, and the frames that mattered most (from the models' attention or from Grad-CAM heat maps).
In: the answer and every model's view **Out:** a short explanation
Why it is there: an answer you can check is worth more than one you have to trust. It also shows *where* the agent is unsure, e.g. between a pull and a hook.

2 The maths, simply

Scores to probabilities. A model gives one score s_c to each shot c. The softmax turns scores into probabilities that are positive and add up to 1:

$$p(c) \;=\; \frac{e^{s_c}}{\sum_{k=1}^{15} e^{s_k}}.$$

A higher score gives a higher probability.

Confidence. The confidence is simply the biggest probability, and the answer is the shot that has it:

$$\text{confidence} = \max_c \, p(c), \qquad \text{answer} = \arg\max_c \, p(c).$$

The gate.

$$\text{go to the experts} \iff \max_c \, p_{\text{fast}}(c) < \tau.$$

The weighted vote. With experts $1, \ldots, K$ and weights $w_1, \ldots, w_K$ that are positive and add up to 1:

$$p_{\text{fused}}(c) \;=\; \sum_{k=1}^{K} w_k \, p_k(c), \qquad \sum_{k=1}^{K} w_k = 1.$$

If every weight is the same, this is a plain average.

The status.

$$\text{status} = \begin{cases} \texttt{cannot_classify} & \text{the batter was not seen enough,} \\ \texttt{low_confidence} & \text{even the fused confidence is below } \tau, \\ \texttt{ok} & \text{otherwise.} \end{cases}$$

A `low_confidence` answer is still given, but it is marked, so nobody mistakes it for a sure one.

3 A worked example

The numbers below are made up, only to show how the steps fit together. They are not results.

1. A clip comes in. The batter is found in 14 of 15 frames, so the quality check passes.
2. The fast model says: Cover Drive 0.62, Straight Drive 0.21, Lofted Offside 0.09, ... Its confidence is 0.62.
3. With $\tau = 0.9$, $0.62 < 0.9$: the agent is unsure and asks the four experts.
4. The experts say Cover Drive with 0.71, 0.55, 0.48 and 0.66; two of them put Straight Drive second.
5. With weights $0.4, 0.2, 0.2, 0.2$, the fused probability of Cover Drive is $0.4 \times 0.71 + 0.2 \times (0.55 + 0.48 + 0.66) = 0.62$, the highest of the 15. That is still below τ, so the answer is Cover Drive with status `low_confidence`.
6. The explanation: "Cover Drive (0.62), then Straight Drive and Lofted Offside. All four experts agree on Cover Drive. The bat's path in frames 8 to 11 mattered most."

4 How it is set up (all on validation data)

Three things are chosen, and all of them on the validation set, never on the test set:

1. **Which models.** The fast model is the best small one; the experts are the best model from each *different* way of seeing (motion, frame by frame, body, movement only).
2. **The weights** w_k: the ones that give the best combined result on validation clips.
3. **The threshold** τ: the lowest one at which the agent is still as good as asking every expert every time. Lower τ means fewer clips need the experts, so the agent is faster.

The test set is used once, at the very end, to report how well the finished agent works.

5 What the agent returns

Field	What it holds
`shot`	the answer: one of the 15 shots
`confidence`	the probability of that shot
`top3`	the three most likely shots, with probabilities
`status`	`ok`, `low_confidence` or `cannot_classify`
`models`	which models were asked, and what each one said
`key_frames`	the frames that mattered most
`track`	every kept frame's number in the raw clip and its crop box; the 15 used by the models are marked

Table 1: The agent's output, the same for every clip.

6 Why it is built this way

- **Fast on easy clips:** most clips stop at the fast model.
- **Careful on hard clips:** several different views vote.
- **Honest:** it refuses clips it cannot read, and marks answers it is not sure of.
- **Checkable:** every answer comes with its reasons.
- **Simple:** each step is a plain function around a trained model; there is nothing to prompt and nothing random in the decisions.

7 Where it lives

The agent is built in the CricShot project after its final phase, when the models, the weights and τ are known. It is then handed to CricketMind as one LangGraph subgraph, which becomes CricketMind's classifier node. CricketMind gives it a raw clip and gets back the output in the table above. CricketMind describes the agent's design, but never reports its accuracy: those numbers belong to the CricShot accuracy paper.

Words used here

Agent a program that takes a sequence of steps and makes decisions between them; here, a fixed flowchart.

Node one step in the flowchart.

LangGraph a Python library for building such flowcharts.

YOLO a fast object detector, used to find the batter.

GRU a small network that reads a sequence (here, 15 frames) in order and remembers what it has seen.

3D CNN a network that looks at a short video as a block of frames, so it sees motion directly.

Pose the positions of 33 body points (shoulders, elbows, hips, knees, ...) in each frame.

Optical flow for every pixel, how far and in which direction it moved since the frame before.

Softmax the formula that turns scores into probabilities.

Validation set clips set aside to make choices with; the test set is kept for the final, honest measurement.